Objective(s):

a. To be able to implement binary-search-tree delete(BNode n, int d) method
b. To be able to implement binary tree rotation method
c. To be able to find the inorder predecessor and successor in a binary search tree.

**Task 1:** Complete MyBST_XXYYYY.java which extends MyBST_Basic_XXYYYY.

- Complete BNode getRoot() **in** MyBST_XXYYYY since we will send the tree to a static method.
- Complete findMin(BNode n) and findMax(BNode n)

```
package pack;

public class MyBST_XXYYYY extends MyBST_Basic_XXYYYY {
  public MyBST_XXYYYY() { super() };
  public MyBST_XXYYYY(Integer[] input) {
      super(input);
  }
  public BNode getRoot() {
      /* your code 1 */;
  }
  public BNode findMin(BNode node) {
      if (node == null) throw new IllegalArgumentException("Empty Tree");
      while (node.left != null) {
          node = node.left;
      }
      return node;
  }
  public BNode findMax(BNode node) {
      if (node == null) throw new IllegalArgumentException("Empty Tree");
      while (/* your code 2 */) {
          node = node.right;
      }
      return node;
  }
```

```
  public static void demo1() {
    System.out.println("---- find min & max ----");
    Integer data[] = {45, 12, 30, 55, 31, 64, 59};
    MyBST_XXYYYY bst = new MyBST_XXYYYY(data);

    System.out.println(bst.findMin(bst.getRoot())); // null <- 12 -> 30
    System.out.println(bst.findMax(bst.getRoot())); // 59 <- 64 -> null

    bst.insert(70);
    System.out.println(bst.findMax(bst.getRoot())); // null <- 70 -> null
  }
```

**Task 2:** Complete `delete(BNode n, int d)`

```
public static void demo2() {
    System.out.println("---- delete ----");
    Integer data[] = {15, 34, 20, 10, 18, 16, 12, 8};
    MyBST_XXYYYY bst = new MyBST_XXYYYY(data);
    System.out.println(bst); // 8 10 12 15 16 18 20 34
    System.out.println(bst.search(10)); // 8 <- 10 -> 12
    System.out.println(bst.search(15)+"\n"); // 10 <- 15 -> 34

    bst.delete(bst.getRoot(), 34);
    bst.delete(bst.getRoot(), 10);
    bst.delete(bst.getRoot(), 100);
    System.out.println(bst); // 8 12 15 16 18 20
    System.out.println(bst.search(10)); // null
    System.out.println(bst.search(15)); // 8 <- 15 -> 20
    System.out.println(bst.search(20)); // 18 <- 20 -> null
}
```

**Task 3:** Inorder Predecessor and Successor

In a binary search tree, an inorder traversal visits node values in ascending order.

A full inorder traversal may visit every node before finding the answer, requiring O(n) time in the worst case. Do not perform a full inorder traversal or build an inorder list for this task.

A directed recursive solution is possible: compare d with the current node and continue to only one child on each call. This follows at most one root-to-leaf path and runs in O(h) time, where h is the height of the tree. However, recursion uses O(h) space for the call stack.

For this task, use an iterative approach. Apply the BST ordering property to move left or right from the root while keeping one additional candidate variable for the current predecessor or successor. This approach also runs in O(h) time and uses O(1) extra space.

Complete these methods in MyBST_XXYYYY:

```
BNode inorderPredecessor(int d)
BNode inorderSuccessor(int d)
```

- The inorder predecessor of d is the node visited immediately before the node containing d in an inorder traversal. It is the largest value smaller than d.
- The inorder successor of d is the node visited immediately after the node containing d in an inorder traversal. It is the smallest value larger than d.
- Return null when no predecessor or successor exists.
- You may assume that d exists in the BST and that all values are distinct.
- Use the BST ordering property; do not modify the tree. Your solution should run in O(h) time, where h is the height of the tree.

```
public static void demo3() {
  System.out.println("---- find predecessor & successor ----");
  Integer data[] = {32, 46, 18, null, 43, 25, 31, 13};
  MyBST_XXYYYY bst = new MyBST_XXYYYY(data);

  System.out.println(bst.inorderPredecessor(31)); // null <- 25 -> 31
  System.out.println(bst.inorderSuccessor( 43)); // 43 <- 46 -> null
  System.out.println(bst.inorderPredecessor( 13)); // null
}
```

**Task 4:** Complete

- static void leftRotate(MyBST_XXYYYY bst, BNode node)
- static void rightRotate(MyBST_XXYYYY bst, BNode node)

  // static void rightLeftRotate(MyBST_XXYYYY bst, BNode node) is provided

```
public static void demo4() {
    System.out.println("---- rotate ----");
    Integer[] data1 = {4,5,6,9};
    MyBST bst1 = new MyBST(data1);
    System.out.println(bst1.search(5)); // null <- 5 -> 6
    System.out.println(bst1.search(6)); // null <- 6 -> 9
    System.out.println(bst1.search(9)); // null <- 9 -> null

    MyBST.leftRotate(bst1, bst1.search(5));
    System.out.println(bst1.search(5)); // null <- 5 -> null
    System.out.println(bst1.search(6)); // 5 <- 6 -> 9
    System.out.println(bst1.search(9) + "\n"); // null <- 9 -> null

    Integer[] data2 = {2,4,7,5,6};
    MyBST bst2 = new MyBST(data2);
    System.out.println(bst2.search(4)); // null <- 4 -> 7
    System.out.println(bst2.search(7)); // 5 <- 7 -> null
    System.out.println(bst2.search(5)); // null <- 5 -> 6

    MyBST.rightLeftRotate(bst2, bst2.search(4));
    System.out.println(bst2.search(4)); // null <- 4 -> null
    System.out.println(bst2.search(7)); // 6 <- 7 -> null
    System.out.println(bst2.search(5)); // 4 <- 5 -> 7
}
```

**Submission:** Your **MyBST_XXYYYY.java** where. XX are the first two digits of your student id and YYYY are the last four.

Due date: TBA